

School of Science and Engineering

AI624 AI on Edge Devices

ASSIGNMENT 2 – Pruning

Due Date: 11:59 PM, 28th September.

Format: 4 tasks worth a total of 100 marks

Instructions:

- Collaboration with peers is allowed, but copying another student's solution is strictly prohibited. This is not a group assignment—each student must submit their own work.
 - Submit all scripts/notebooks along with visible results, as well as a comprehensive Markdown file summarizing the results and analysis for each task and subtask.
 - Submit one ZIP file containing a main Jupyter notebook with all outputs visible, any supporting Python modules, and a Markdown report summarizing results and analysis for every task. Present results in tables where appropriate and equations in LaTeX. Include dependencies, execution instructions, and a brief codebase outline in a README.
 - Submit your work on LMS by the deadline.
 - Start early. This is a lengthy assignment, and we do not expect you to complete it in the last few days.
 - If you have any concerns, contact the instructor or teaching assistants. You are also encouraged to use the assignment Slack channel for questions and discussion.
- You are among the most competent individuals in the country. Do not let plagiarism undermine your learning. Any instance of plagiarism will be dealt with according to the university's rules and regulations.
 - You must acknowledge any use of generative AI tools. Include the following statement where applicable:
I have used [insert Tool Name] to [write, generate, plot, or compute; explain the specific use of generative AI] [number of times].
 - You may optionally share a link to the relevant chatbot conversation for further clarity.

1 Task 0: Load and Profile the Base Model

In this task, you will load the pretrained model and establish the baseline used throughout the assignment.

Load a pretrained ResNet-18 model from https://github.com/huyvnphan/PyTorch_CIFAR10, using the architecture in `cifar10_models/resnet.py` and the checkpoint `resnet18.pt` from the repository's linked weights archive. Alternative sources are provided in the references; you may also explore other repositories.

To understand how profiling for a model works, look into the references section, specifically PyTorch's profiling documentation for assistance.

Implementation Procedure

1. Load the pretrained ResNet-18 model and verify that the checkpoint is loaded correctly.
2. Load the CIFAR-10 training and test datasets.
3. Apply the preprocessing expected by the pretrained model.
4. Construct the training and test dataloaders.
5. Report the batch size, input resolution, normalization, and data augmentation used.
6. Evaluate the full-precision model on the training and test-set and report performance metrics highlighted in the deliverables section for this task.
7. Profile the model using the same batch size and hardware (same local machine ideally) that will be used for all later experiments.

Deliverables

Report:

- Train Top-1 accuracy
- Test Top-1 accuracy
- Top-5 accuracy
- Macro-F1 score
- Number of model parameters
- Serialized model size in MB
- MACs
- FLOPs
- Average inference latency in milliseconds
- Peak GPU memory usage
- Average GPU memory usage
- CPU and GPU energy consumption

Average latency, memory, and energy measurements over several batches. Use the same profiling procedure for every model in the remaining tasks.

For Tasks 1 and 2, select a final target sparsity between approximately 70% and 90%. Aim to keep the final accuracy drop within approximately 5–15 percentage points of the dense baseline, or better.

2 Task 1: Unstructured Post-Training Pruning

In this task, you will apply unstructured magnitude-based pruning to the pretrained ResNet-18 model. Unstructured pruning removes individual weights while keeping the original tensor dimensions unchanged. You may use the pre-trained model via the links mentioned in Task 0 or the references section as well as explore alternatives on either Github or the internet. Kindly ensure that you use the same architecture across all tasks, and the same dense pretrained checkpoint for Tasks 0, 1, and 3 to maintain consistency and fair comparison of results when required.

Notation

Let the weight tensor of layer ℓ be

$$W^{(\ell)}.$$

Define a binary pruning mask

$$M^{(\ell)} \in \{0, 1\}^{\text{shape}(W^{(\ell)})}.$$

The pruned weight tensor is

$$\widetilde{W}^{(\ell)} = M^{(\ell)} \odot W^{(\ell)}.$$

A mask value of zero indicates that the corresponding weight has been pruned.

For magnitude pruning, the importance of weight i is

$$I_i^{(\ell)} = |W_i^{(\ell)}|.$$

Weights with the smallest absolute values are pruned.

Implementation Procedure

1. Layer-wise sensitivity analysis

For every convolutional and linear layer, independently apply the following sparsity levels:

$$\{0, 10, 20, 50, 70, 80, 90\}\%.$$

For every layer and sparsity value:

- Start from the original dense checkpoint.
- Prune only the selected layer.
- Keep all other layers dense.
- Evaluate the model without fine-tuning.
- Restore the original model before testing the next layer.

Plot model accuracy against sparsity for every layer.

2. Local magnitude pruning

Based on the sensitivity curves, select a separate sparsity ratio for each layer.

Assign lower sparsity to sensitive layers and higher sparsity to layers that tolerate pruning.

The overall sparsity is

$$s_{\text{overall}} = 1 - \frac{\sum_{\ell} \|M^{(\ell)}\|_0}{\sum_{\ell} N_{\ell}},$$

where N_{ℓ} is the number of eligible weights in layer ℓ .

3. Global magnitude pruning

Collect all eligible convolutional and linear weights and determine one global magnitude threshold.

Apply global pruning at the same overall sparsity used for local pruning.

After pruning, report the sparsity automatically assigned to each layer.

Verify that no layer has been completely pruned.

4. Fine-tune the pruned models

Fine-tune both the locally pruned and globally pruned models.

Keep all pruning masks fixed during fine-tuning.

After every optimizer step, reapply the masks:

$$W^{(\ell)} \leftarrow M^{(\ell)} \odot W^{(\ell)}.$$

Use the same optimizer, number of epochs, learning-rate schedule, batch size, and random seed for both models.

5. Verify the masks

For every pruned layer:

- Verify that all masked positions contain zero.
- Compare the sparsity calculated from the mask with the sparsity measured directly from the weight tensor.
- Confirm that pruned weights remain zero after fine-tuning.

6. Sparse storage

Convert the pruned weight tensors to COO format using `torch.sparse`.

Save:

- Nonzero values
- Indices
- Tensor shapes
- Pruning masks

7. Sparse inference

Profile the dense masked model and the COO representation.

If a sparse convolutional tensor must be converted back to dense before convolution, state this clearly.

Use sparse matrix multiplication for linear layers where supported.

Experiments

Compare:

1. Dense full-precision baseline
2. Local magnitude pruning
3. Global magnitude pruning
4. Dense masked inference
5. Sparse COO storage and inference, where supported

Use the same final overall sparsity for local and global pruning.

Deliverables

For local and global pruning, report:

- Overall sparsity
- Layer-wise sparsity
- Train and Test Top-1 accuracy
- Train and Test Top-5 accuracy
- Macro-F1 score
- Dense model size
- Sparse model size including indices
- MACs
- FLOPs
- Inference latency
- Peak GPU memory usage
- Average GPU memory usage
- CPU and GPU energy consumption
- Training and validation curves

Required Discussion

Discuss:

- Which layers were the most sensitive to pruning?
- Did global pruning produce a reasonable layer-wise sparsity distribution?
- How much accuracy was recovered through fine-tuning?
- Did COO storage reduce the actual model size?
- Why can a model containing many zero-valued weights still have nearly the same latency as the dense model?

3 Task 2: Saliency-Based Iterative Pruning

In this task, you will apply pruning gradually during training using the GraSP saliency criterion.

Unlike magnitude pruning, GraSP assigns importance based on the effect of removing a weight on gradient flow. We would recommend going through the paper **"Picking Winning Tickets Before Training By Preserving Gradient Flow"** attached in the references section to understand algorithm 1 in the paper before attempting this task. For iterative magnitude and iterative GraSP pruning, use the same ResNet-18 architecture and repository as Tasks 0 and 1, but initialize the model with random weights using `resnet18(pretrained=False)`. Train this model from scratch and do not load the pretrained checkpoint. Both iterative methods must start from identical randomly initialized weights and use the same training budget.

Notation

Let the model parameters be

$$\boldsymbol{\theta} = \{\theta_1, \theta_2, \dots, \theta_P\}.$$

For a calibration batch, calculate the loss

$$\mathcal{L}(\boldsymbol{\theta}).$$

The gradient is

$$\mathbf{g} = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}).$$

GraSP uses the Hessian-gradient product

$$\mathbf{Hg} = \nabla_{\boldsymbol{\theta}}^2 \mathcal{L}(\boldsymbol{\theta}) \mathbf{g}.$$

Do not construct the Hessian explicitly. Compute the Hessian-gradient product using automatic differentiation.

The GraSP score for parameter θ_i is

$$S_i = -\theta_i (\mathbf{Hg})_i.$$

Follow the ranking and pruning rule given in the GraSP paper.

Implementation Procedure

1. Initialize and warm up the model

Initialize ResNet-18 with random weights.

Train the model until it reaches approximately 20% accuracy.

Save the initial weights, warm-up checkpoint, optimizer state, and training curves.

2. Prepare the calibration set

Select a small subset of the training data for computing the GraSP scores.

Report:

- Number of calibration samples
- Calibration batch size

- Number of batches

If several batches are used, accumulate or average the gradients consistently.

3. Compute GraSP scores

Use `torch.autograd.grad` to compute the gradients and Hessian-gradient product.

Do not prune bias or normalization parameters.

Verify the sorting direction before constructing the mask.

4. Apply iterative pruning

Let the final target sparsity be s^* .

Apply pruning in three stages:

$$s_1 = 0.50s^*, \quad s_2 = 0.75s^*, \quad s_3 = s^*.$$

Use the following schedule:

- At approximately 20% accuracy, prune to 50% of the final target.
- Continue training.
- At approximately 40% accuracy, prune to 75% of the final target.
- Continue training.
- At approximately 60% accuracy, prune to the final target.
- Continue training until convergence or until the training budget is exhausted.

These accuracy values are not strict. Any changes must be reported and justified.

5. Maintain cumulative masks

If $M_k^{(\ell)}$ is the mask introduced at pruning stage k , update the cumulative mask as

$$M_{\text{cum},k}^{(\ell)} = M_{\text{cum},k-1}^{(\ell)} \odot M_k^{(\ell)}.$$

A weight pruned in an earlier stage must remain pruned.

Reapply the cumulative mask after every optimizer step.

6. Profile the final model

Convert the final pruned tensors to COO format and profile the model using the same procedure as Task 1.

Experiments

Compare:

1. Magnitude pruning from the first task.
2. Iterative magnitude pruning
3. Iterative GraSP pruning

Use the same:

- Initial weights
- Final target sparsity

- Number of training epochs
- Optimizer
- Learning-rate schedule
- Batch size

Deliverables

For every pruning method, report:

- Accuracy before and after every pruning stage
- Sparsity after every pruning stage
- Final train and test accuracy
- Macro-F1 score
- Training and validation loss curves
- Training and validation accuracy curves
- Layer-wise remaining-weight ratio
- Model size
- Inference latency
- Peak and average GPU memory usage
- CPU and GPU energy consumption
- Time required to calculate the pruning criterion

Required Discussion

Discuss:

- Did iterative pruning outperform magnitude-based pruning?
- Did GraSP perform better than iterative magnitude pruning?
- Did any layer collapse at high sparsity?

4 Task 3: Structured Pruning

Regression-Based Channel Pruning

In this task, you will implement structured pruning (channels) of convolutional layers using the regression-based method introduced in He et al., 2017 on ResNet-18, starting from the same pretrained checkpoint used in Tasks 0 and 1.

The pruning algorithm can be divided into the following steps:

1. Collecting input–output pairs.

For a given convolutional layer with weights

$$W \in \mathbb{R}^{n \times c \times k_h \times k_w}$$

and input feature maps

$$X \in \mathbb{R}^{N \times c \times h \times w}$$

(the inputs for each layer will be collected using a small calibration dataset):

- (a) Using `torch.nn.functional.unfold`, extract all $k_h \times k_w$ sliding windows from X . After unfolding, reshape so that

$$X_{\text{unf}} \in \mathbb{R}^{(N \cdot L) \times (c \cdot k_h \cdot k_w)},$$

where

$$L = h_{\text{out}} \times w_{\text{out}}$$

is the number of output positions.

- (b) Reshape the convolutional filters to

$$W_{\text{mat}} \in \mathbb{R}^{(c \cdot k_h \cdot k_w) \times n}.$$

- (c) Compute the output activations as

$$Y = X_{\text{unf}} W_{\text{mat}} \in \mathbb{R}^{(N \cdot L) \times n}.$$

2. Subproblem (i): Solve for β (channel selection).

- (a) For each input channel i , define

$$Z_i = X_i W_i^\top, \quad Z_i \in \mathbb{R}^{(N \cdot L) \times n},$$

where X_i is the slice of X_{unf} corresponding to channel i and W_i is the slice of W for that channel.

- (b) Formulate the LASSO regression problem:

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2N} \left\| Y - \sum_{i=1}^c \beta_i Z_i \right\|_F^2 + \lambda \|\beta\|_1.$$

- (c) We can solve this problem using an off-the-shelf LASSO solver (e.g., `sklearn.linear_model.Lasso`). Channels with $\beta_i = 0$ are pruned (or more simply, just set the k lowest non-zero values of β to zero). Continue adjusting λ until only c_0 channels remain.

3. Subproblem (ii): Solve for W (weight reconstruction).

- (a) Construct the reduced design matrix

$$X' = [\beta_1 X_1, \beta_2 X_2, \dots, \beta_{c_0} X_{c_0}] \in \mathbb{R}^{(N \cdot L) \times (c_0 \cdot k_h \cdot k_w)}.$$

- (b) Solve the least-squares regression problem

$$W' = \arg \min_{W'} \left\| Y - X' W'^\top \right\|_F^2.$$

This can be solved using `torch.linalg.lstsq`.

- (c) Reshape W' back to convolutional form (n, c_0, k_h, k_w) .

While implementing, keep the following instructions in mind:

1. Apply pruning sequentially, starting from the second convolutional block.
2. Maintain the same sparsity ratio as decided in Task 1 (via sensitivity analysis). Alternatively, you can adjust the ratio for each layer on an ad-hoc basis.
3. You will first apply step i from the paper iteratively (or in a single iteration) until the target sparsity is reached for the layer.
4. Remember: pruning input channels in one layer requires pruning corresponding output channels in the previous layer. This means you will need to achieve a lower sparsity than your target in the current layer, as some sparsity will be compensated for by pruning the next layer.
5. When removing channels, update the corresponding BatchNorm layers and any affected short-cut projections. Ensure that both branches of every residual addition retain matching channel dimensions and consistent channel indices.
6. Remember to adjust the weights W for each layer after its pruning based on the least square solution as in the paper (step ii).
7. Continue pruning through the network, layer by layer. You can still prune the linear layers for this task using unstructured pruning as before. However, it is not necessary. You still need to meet the overall sparsity ratio, consistent with the previous tasks.
8. Finetune for a final time over a few epochs based on the task loss (Cross Entropy) after all pruning is done.
9. Profile as done in previous tasks and report results in the table. We don't need any specialized libraries for sparse convolution here, since the remaining filters are dense.

Deliverables

- Record results in a table in the markdown file.
- You may decide the number of fine-tuning epochs depending on your compute budget and required accuracy.
- Did you see an improvement in performance (memory, latency) while meeting the accuracy constraint? Why or why not?

References

1. S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," ICLR, 2016. <https://arxiv.org/abs/1510.00149>
2. N. Lee, T. Ajanthan, and P. H. S. Torr, "SNIP: Single-shot Network Pruning Based on Connection Sensitivity," ICLR, 2019. <https://arxiv.org/abs/1810.02340>
3. C. Wang, G. Zhang, and R. Grosse, "Picking Winning Tickets Before Training by Preserving Gradient Flow," ICLR, 2020. <https://arxiv.org/abs/2002.07376>
4. A. Mishra et al., "Accelerating Sparse Deep Neural Networks," 2021. <https://arxiv.org/abs/2104.08378>
5. G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards Any Structural Pruning," CVPR, 2023. <https://arxiv.org/abs/2301.12900>
6. Y. He, J. Zhang, and J. Sun, "Channel Pruning for Accelerating Very Deep Neural Networks," ICCV, 2017. <https://arxiv.org/abs/1707.06168>

7. PyTorch, “Semi-Structured Sparsity.” https://docs.pytorch.org/tutorials/advanced/semi_structured_sparse.html
8. PyTorch, “Sparse Tensor Documentation.” <https://docs.pytorch.org/docs/stable/sparse.html>
9. Han Lab, MIT, “Hardware-Aware Efficient Deep Learning.” <https://hanlab.mit.edu/courses>
10. ResNet-18 Pretrained Models:
<https://github.com/kuangliu/pytorch-cifar>
<https://github.com/huggingface/pytorch-image-models>
11. PyTorch Profiler Documentation:
https://docs.pytorch.org/tutorials/recipes/recipes/profiler_recipe.html
<https://docs.pytorch.org/docs/stable/profiler.html>